

Yedekli Telemetri Sistemi – STM32 Tabanlı FreeRTOS Uygulaması

Bu proje, kamikaze drone platformları gibi kritik görev yükü taşıyan otonom sistemlerde görev sırasında sensör verilerinin ve sistem sağlık bilgilerinin yedekli olarak iletimini sağlamak amacıyla geliştirilmiştir. Gerekli verileri toplayarak hem CAN hem de UART üzerinden senkron şekilde iletir. Tüm uygulama FreeRTOS üzerinde çok görevli yapı ile tasarlanmış, görevler fonksiyonel olarak ayrılmış ve donanım arabirimleriyle entegre edilmiştir.

1. Donanım Kullanımı ve Entegrasyon Durumu

STM32 Pin	Fonksiyon	Bileşen	Açıklama
PA15	SPI3_CS	ADIS16475	Slave seçimi – GPIO üzerinden kontrol
PC10	SPI3_SCK	ADIS16475	Saat sinyali
PC11	SPI3_MISO	ADIS16475	Veri çıkışı
PC12	SPI3_MOSI	ADIS16475	Veri girişi
PA0	UART4_TX	Modül	Telemetri veri çıkışı
PA1	UART4_RX	Modül	Telemetri veri girişi
PA11	CAN_RX	MCP2551	CAN veri alma
PA12	CAN_TX	MCP2551	CAN veri gönderme
ADC1_IN18	Sıcaklık Sensörü	Dahili	Sıcaklık ölçümü

2. FreeRTOS Görev Yapısı

Görev İsmi	Görev Tanımı
SensorTask	SPI3 üzerinden IMU'dan gyro ve ivme verilerini okur.
TempTask	ADC üzerinden dahili sıcaklık sensöründen veri alır.
CommsTask	Struct içindeki verileri CAN ve UART ile iletir.
ErrorMonitorTask	IMU SPI bağlantısı ve sıcaklık değerlerini denetler

3. Telemetri Mesaj Formatı

Her gönderilen mesaj, aşağıdaki formata uygun şekilde hem UART hem de CAN üzerinden aktarılır:

Alan	Açıklama	Kodda Karşılığı
0xDD	Mesaj başlığı	Sabit: 0xDD
timestamp	Sistem zamanı (saniye)	imu_data.systick_counter
temp_degC	Sıcaklık (x100 formatında)	imu_data.temperature
gyro_x/y/z	Gyroskop verileri	imu_data.gyro_x/y/z
accel_x/y/z	İvmeölçer verileri	imu_data.accel_x/y/z
error_flags	Hata bayrakları	imu_data.error_flags
port_id	Gönderim portu (0x01: UART, 0x02: CAN)	imu_data.port_id

4.Kullanılan Veri Yapıları ve Değişkenler

Proje içerisinde, sensör verilerini toplamak, sistem durumunu izlemek ve görevler arası veri bütünlüğünü sağlamak amacıyla struct ve global değişkenler tanımlanmıştır.

Struct Yapısı

```
typedef struct {
    int16_t temperature;
    uint16_t timestamp;
    uint8_t error_flags;
    uint8_t port_id;
    int32_t gyro_x;
    int32_t gyro_y;
    int32_t gyro_z;
    int32_t accel_x;
    int32_t accel_y;
    int32_t accel_z;
} IMU_Data_t;
```

Telemetriye iletilecek veriler, erişilebilirliği artırmak ve kod okunabilirliğini optimize etmek amacıyla yapılandırılmış veri yapısı içerisinde tutulmaktadır. Bu yöntem, hem sistem entegrasyonunu kolaylaştırmakta hem de bakım süreçlerinde yüksek izlenebilirlik (traceability) sağlamaktadır. Tanımlanan yapı, toplamda 30 baytlık kompakt bir veri bloğu oluşturarak haberleşme protokolleri açısından bant genişliği kullanımını minimize ederken, veri bütünlüğünü de koruma altına almaktadır.

```
imu_data.gyro_x = ((int32_t)ADIS_ReadRegister(0x06) << 16) | ADIS_ReadRegister(0x04);
imu_data.gyro_y = ((int32_t)ADIS_ReadRegister(0x0A) << 16) |
ADIS_ReadRegister(0x08);
imu_data.gyro_z = ((int32_t)ADIS_ReadRegister(0x0E) << 16) |
ADIS_ReadRegister(0x0C);

imu_data.accel_x = ((int32_t)ADIS_ReadRegister(0x12) << 16) |
ADIS_ReadRegister(0x10);
imu_data.accel_y = ((int32_t)ADIS_ReadRegister(0x16) << 16) |
ADIS_ReadRegister(0x14);
imu_data.accel_z = ((int32_t)ADIS_ReadRegister(0x1A) << 16) |
ADIS_ReadRegister(0x18);
```

ADIS16475'in datasheetinde tanımlanan register haritası referans alınarak, her bir eksene (X, Y, Z) karşılık gelen sensör verileri doğrudan bellek eşlemeli register'lerden okunmaktadır. Struct yapısına dinamik olarak entegre edilmektedir. Bu yaklaşım, hem eksen bazlı ayrıştırmayı (axis-specific parsing) mümkün kılmakta hem de veri işleme sürecinde yüksek doğruluk ve sistematik erişilebilirlik sağlamaktadır.

```
CAN_TxHeaderTypeDef TxHeader;
uint32_t TxMailbox;
uint8_t TxData[8];

TxHeader.DLC = 8;
TxHeader.IDE = CAN_ID_STD;
TxHeader.RTR = CAN_RTR_DATA;
TxHeader.StdId = 0x123;
TxHeader.TransmitGlobalTime = DISABLE;

uint8_t* data_ptr = (uint8_t*)&imu_data;

// 31 byte → 4 frame (8+8+8+6)
for (uint8_t i = 0; i < 31; i += 8)
{
    uint8_t size = (i + 8 <= 30) ? 8 : (30 - i); // son parça 6 byte
    memset(TxData, 0, 8); // kalan byte'ları sıfırla
    memcpy(TxData, data_ptr + i, size);
    HAL_CAN_AddTxMessage(&hcan1, &TxHeader, TxData, &TxMailbox);
    HAL_Delay(1); // MCP2551 gibi transceiver'lar için gerekli olabilir
}
```

Bu kod bloğu, imu_data yapısında bulunan 30 baytlık IMU verisini, CAN protokolü üzerinden parçalara bölerek gönderir. CAN protokolü, her bir çerçevede en fazla 8 bayt veri taşıyabildiği için, bu veri dört ayrı çerçeve halinde iletilir (8 + 8 + 8 + 6 bayt). Öncelikle, TxHeader yapısı standart kimlik (Standard ID) modunda (CAN_ID_STD) ve veri çerçevesi (CAN_RTR_DATA) olarak yapılandırılır. Mesajın gönderileceği hedef kimlik (ID) StdId alanına atanır (0x123), ve zaman damgası gönderimi devre dışı bırakılır.

Ardından, imu_data yapısının bellekteki adresi bir bayt dizisine (uint8_t*) çevrilerek düz veri bloğu şeklinde erişim sağlanır. Döngü içinde, veri 8 baytlık parçalara ayrılarak TxData tamponuna kopyalanır. Eğer kalan veri 8 bayttan azsa, sadece kalan kadar bayt kopyalanır ve tamponun geri kalanı sıfırlanır. Her bir parça HAL_CAN_AddTxMessage fonksiyonu ile CAN veri hattına iletilir.

```
char uart_buf[150];
int len = snprintf(uart_buf, sizeof(uart_buf),
    "T:%u E:%u P:%u GX:%ld GY:%ld GZ:%ld AX:%ld AY:%ld AZ:%ld\n",
    imu_data.timestamp,
    imu_data.error_flags,
    imu_data.port_id,
    imu_data.gyro_x,
    imu_data.gyro_y,
    imu_data.gyro_z,
    imu_data.accel_x,
    imu_data.accel_y,
    imu_data.accel_z,
    imu_data.temperature);
HAL_UART_Transmit(&huart4, (uint8_t*)uart_buf, len, HAL_MAX_DELAY);
```

Bu kod bloğu, imu_data yapısında tutulan sensör verilerini okunabilir bir metin formatında UART üzerinden iletmek için kullanılır. snprintf fonksiyonu ile zaman damgası, hata durumu, port ID, jiroskop ve ivmeölçer eksen verileri ile sıcaklık bilgisi uart_buf tamponuna formatlanarak yazılır. Daha sonra, bu metin verisi HAL_UART_Transmit fonksiyonu ile UART4 üzerinden gönderilir.

5. Hata Senaryoları ve İzleme

```
uint8_t error = 0x00;
uint16_t diag_status = ADIS_ReadRegister(0x02); // IMUDAKİ HATAYI VERECEK REGISTER

int16_t temp = imu_data.temperature;
float temp_c = temp / 100.0f;

if (diag_status != 0x0000 && (temp_c < -40.0f || temp_c > 125.0f))
{
    error = 0x11; // İkisi birden hata varsa
}
else if (diag_status != 0x0000)
{
    error = 0x01; // Sadece IMU SPI hatası
}
else if (temp_c < -40.0f || temp_c > 125.0f)
{
    error = 0x10; // Sadece sıcaklık hatası
}
else
{
    error = 0x00; // Hata yok
}

imu_data.error_flags = error;
```

Bu kod bloğu, IMU sensöründen alınan bilgi ve sıcaklık verisine göre sistemde hata kontrolü yapar. Öncelikle, IMU'nun 0x02 adresli register'ından hata durumu okunur ve sıcaklık verisi santigrat dereceye çevrilirerek kontroller yapılır. Hata durumları struct yapısına yazılır.

Hata Durumu	error_flags Değeri
IMU Hatası	0x01
Aşırı/Düşük Sıcaklık	0x10
Her iki hata birlikte	0x11
Hata Yok	0x00

6. Organizasyonu

